
Спецификация

Компонент Eco.StdLib.C89

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

Авторы	Компания
Владимир Башев	ПИРФ

Содержание

1. Обзор	4
1.1. Введение	4
1.2. Примечание	4
1.3. Ссылки	4
2. Компонент Eco.StdLib.C89	5
2.1. IEcoStdLibC89 описание на ECO IDL	6
2.1.1. Функция atof	8
2.1.2. Функция atoi	8
2.1.3. Функция atol	8
2.1.4. Функция strtod	8
2.1.5. Функция strtol	8
2.1.6. Функция strtoul	8
2.1.7. Функция rand	9
2.1.8. Функция srand	9
2.1.9. Функция calloc	9
2.1.10. Функция free	9
2.1.11. Функция malloc	9
2.1.12. Функция realloc	9
2.1.13. Функция abort	10
2.1.14. Функция atexit	10
2.1.15. Функция exit	10
2.1.16. Функция getenv	10
2.1.17. Функция system	10
2.1.18. Функция bsearch	10
2.1.19. Функция qsort	11
2.1.20. Функция abs	11
2.1.21. Функция div	11
2.1.22. Функция labs	11
2.1.23. Функция ldiv	11
2.1.24. Функция mblen	11
2.1.25. Функция mbtowc	11
2.1.26. Функция wctomb	12

2.1.27.	Функция mbstowcs.....	12
2.1.28.	Функция wcstombs.....	12
Приложение А: Обучающие программы		13

1. Обзор

Данный документ описывает требования к реализации компонента Eco.StdLib.C89 (Компонент стандартная библиотека).

1.1. Введение

Описание.

1.2. Примечание

- Ключевые слова в документе

1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

2. Компонент Eco.StdLib.C89

Компонент, имеет следующие описание:

2.1. IEcoStdLibC89 описание на ECO IDL

ECO IDL

```
import "IEcoBase1.h"

[
  object,
  uuid(00000000-0000-0000-0000-890000000101),
]

interface IEcoStdLibC89 : IEcoUnknown {

double          atof          ([in] const char* nptr);
int             atoi          ([in] const char* nptr);
long int        atol          ([in] const char* nptr);
double          strtod        ([in] const char* nptr,
                               [in] char** endptr);
long int        strtol        ([in] const char* nptr,
                               [in] char** endptr,
                               [in] int base);
unsigned long int strtoul      ([in] const char* nptr,
                               [in] char** endptr,
                               [in] int base);

int             rand          ();
void            srand         ([in] unsigned int seed);
void*           calloc         ([in] const void* s1,
                               [in] const void* s2,
                               [in] size_t n);
void            free           ([in] const char* s1);
void*           malloc         ([in] const char* s1,
                               [in] const char* s2);
void*           realloc        ([in] const char* s1,
                               [in] const char* s2,
                               [in] size_t n);
void            abort          ([in] char* s1,
                               [in] const char* s2,
                               [in] size_t n);
int             atexit         ([in] const void* s,
                               [in] int c,
                               [in] size_t n);
void            exit           ([in] const char* s,
                               [in] int n);
char*           getenv         ([in] const char* s1,
                               [in] const char* s2);
int             system         ([in] const char* s1,
                               [in] const char* s2);
```

```

void*      bsearch      ([in] const char* s,
                          [in] int c);

void       qsort        ([in] const char* s1,
                          [in] const char* s2);

int        abs          ([in] const char* s1,
                          [in] const char* s2);

div_t      div          ([in] char* s1,
                          [in] const char* s2);

long int   labs         ([in] void* s,
                          [in] int c,
                          [in] size_t n);

div_t      ldiv         ([in] int errnum);

int        mblen        ([in] const char* s);

int        mbtowc       ([in] const char* s);

int        wctomb       ([in] const char* s);

size_t     mbstowcs     ([in] const char* s);

size_t     wcstombs     ([in] const char* s);

}

```

2.1.1. Функция atof

Библиотечная функция C преобразует строковый аргумент `str` в число с плавающей запятой (тип `double`).

2.1.2. Функция atoi

Библиотечная функция C преобразует строковый аргумент `str` в целое число (тип `int`).

2.1.3. Функция atol

Библиотечная функция C преобразует строковый аргумент `str` в длинное целое число (тип `long int`).

2.1.4. Функция strtod

Библиотечная функция C преобразует строку, на которую указывает аргумент `str`, в число с плавающей запятой (тип `double`). Если `endptr` не равно `NULL`, указатель на символ после последнего символа, использованного в преобразовании, хранится в расположении, на которое ссылается `endptr`.

2.1.5. Функция strtol

Библиотечная функция C преобразует начальную часть строки в `str` в длинное целое значение в соответствии с заданным основанием, которое должно быть от 2 до 36 включительно или иметь специальное значение 0.

2.1.6. Функция strtoul

Библиотечная функция C преобразует начальную часть строки в str в беззнаковое длинное целое значение в соответствии с заданным основанием, которое должно быть от 2 до 36 включительно или иметь специальное значение 0.

2.1.7. Функция rand

Библиотечная функция C возвращает псевдослучайное число в диапазоне от 0 до RAND_MAX.

RAND_MAX — это константа, значение которой по умолчанию может варьироваться в зависимости от реализации, но оно должно быть не менее 32767.

2.1.8. Функция srand

Библиотечная функция C void srand(unsigned int seed) заполняет генератор случайных чисел, используемый функцией rand.

2.1.9. Функция calloc

Библиотечная функция C выделяет запрошенную память и возвращает на нее указатель. Разница между malloc и calloc заключается в том, что malloc не устанавливает память в ноль, тогда как calloc устанавливает выделенную память в ноль.

2.1.10. Функция free

Библиотечная функция C освобождает память, ранее выделенную вызовом calloc, malloc или realloc.

2.1.11. Функция malloc

Библиотечная функция C выделяет запрошенную память и возвращает на нее указатель.

2.1.12. Функция realloc

Библиотечная функция `C` пытается изменить размер блока памяти, на который указывает `ptr`, который ранее был выделен с помощью вызова `malloc` или `calloc`.

2.1.13. Функция `abort`

Библиотечная функция `C` прерывает выполнение программы и выходит непосредственно из места вызова.

2.1.14. Функция `atexit`

Библиотечная функция `C` вызывает указанную функцию `func` при завершении программы. Вы можете зарегистрировать свою функцию завершения в любом удобном для вас месте, но она будет вызвана в момент завершения программы.

2.1.15. Функция `exit`

Библиотечная функция `C` немедленно завершает вызывающий процесс. Все открытые файловые дескрипторы, принадлежащие процессу, закрываются, а все дочерние элементы процесса наследуются процессом `1`, `init`, а родительскому процессу посылается сигнал `SIGCHLD`.

2.1.16. Функция `getenv`

Библиотечная функция `C` ищет строку среды, на которую указывает имя, и возвращает связанное значение в строку.

2.1.17. Функция `system`

Библиотечная функция `C` `int system(const char *command)` передает имя команды или имя программы, указанное командой, в хост-окружение для выполнения командным процессором и возвращает его после завершения команды.

2.1.18. Функция `bsearch`

Библиотечная функция `C` ищет по ключу массив объектов `items`, на начальный элемент которых указывает `base`. Размер каждого элемента массива задается параметром `size`.

Содержимое массива должно быть отсортировано по возрастанию в соответствии с функцией сравнения, на которую ссылается `compar`.

2.1.19. Функция `qsort`

Библиотечная функция `C` сортирует массив.

2.1.20. Функция `abs`

Библиотечная функция `C` возвращает абсолютное значение `int x`.

2.1.21. Функция `div`

Библиотечная функция `C` делит `num` (числитель) на `denom` (знаменатель).

2.1.22. Функция `labs`

Библиотечная функция `C` возвращает абсолютное значение `x`.

2.1.23. Функция `ldiv`

Библиотечная функция `C` делит число (числитель) на `denom` (знаменатель).

2.1.24. Функция `mblen`

Библиотечная функция `C` возвращает длину многобайтового символа, на который указывает аргумент `str`.

2.1.25. Функция `mbtowc`

Библиотечная функция C преобразует многобайтовую последовательность в широкий символ.

2.1.26. Функция wctomb

Библиотечная функция C преобразует расширенный символ wchar в его многобайтовое представление и сохраняет его в начале массива символов, на который указывает str.

2.1.27. Функция mbstowcs

Библиотечная функция C преобразует строку многобайтовых символов, на которую указывает аргумент str, в массив, на который указывает pwcs.

2.1.28. Функция wcstombs

Библиотечная функция C преобразует строку pwcs с расширенными символами в многобайтовую строку, начинающуюся с str. В str записывается не более n байт.

Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoStdLibC89
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IdEcoInterfaceBus1.h"
#include "IdEcoStdLibC89.h"
#include "IdEcoStdIO89.h"
#include "IdEcoStringC89.h"
#include "IdEcoTimeC89.h"

/* Прототипы */
bool_t test_atof(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_atoi(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_atol(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_strtod(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_strtol(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_strtoul(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_rand(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoTimeC89* pITime);
bool_t test_srand(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoTimeC89* pITime);
bool_t test_calloc(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_free(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_malloc(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_realloc(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_abort(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_atexit(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_exit(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_getenv(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_system(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO, IEcoStringC89* pIString);
bool_t test_bsearch(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_qsort(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_abs(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_div(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_labs(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);
bool_t test_ldiv(IEcoStdLibC89* pIStdLib, IEcoStdIO89* pIStdIO);

```

```
bool_t test_mblen(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO);
bool_t test_mbtowc(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO, IEcoStringC89* pIString);
bool_t test_wctomb(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO);
bool_t test_mbstowcs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO);
bool_t test_wcstombs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO);
```

```
void ECOCDECLMETHOD functionA (void) {
    printf("This is functionA\n");
}
```

```
int ECOCDECLMETHOD cmpfunc(const void * a, const void * b) {
    return ( *(int*)a - *(int*)b );
}
```

```
int values[] = { 88, 56, 100, 2, 25 };
```

```
#define BUFFER_SIZE 50
```

```
/*
 *
 * <сводка>
 *   Функция EcoMain
 * </сводка>
 *
 * <описание>
 *   Функция EcoMain - точка входа
 * </описание>
 */
int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на системный интерфейс */
    IEcoSystem1* pISys = 0;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */
    IEcoInterfaceBus1* pIBus = 0;
    /* Указатель на интерфейс работы с памятью */
    IEcoMemoryAllocator1* pIMem = 0;
    char_t* name = 0;
    char_t* copyName = 0;
    /* Указатель на тестируемый интерфейс */
    IEcoStdLibC89* pIStdLib = 0;
    /* Указатель на интерфейс стандартного ввода/вывода */
    IEcoStdIOC89* pIStdIO = 0;
    /* Указатель на интерфейс для работы со строкой */
    IEcoStringC89* pIString = 0;
    /* Указатель на интерфейс для работы со временем */
    IEcoTimeC89* pITime = 0;

    /* Проверка и создание системного интрефейса */
    if (pISys == 0) {
        result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&pISys);
        if (result != 0 && pISys == 0) {
            /* Освобождение системного интерфейса в случае ошибки */
            goto Release;
        }
    }

    /* Получение интерфейса для работы с интерфейсной шиной */
```

```

result = pISys->pVTbl->QueryInterface(pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
if (result != 0 || pIBus == 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}
#ifdef ECO_LIB
    /* Регистрация статического компонента для работы со стандартной библиотекой */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdLibC89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053544431);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со стандартным вводом/выводом */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdIOC89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053494F31);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со строкой */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStringC89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000073747231);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со временем */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoTimeC89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_0000000000000000000000000544D4531);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
#endif
    /* Получение интерфейса управления памятью */
    result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void **)&pIMem);

    /* Проверка */
    if (result != 0 && pIMem == 0) {
        /* Освобождение системного интерфейса в случае ошибки */
        goto Release;
    }

    /* Выделение блока памяти */
    name = (char_t *)pIMem->pVTbl->Alloc(pIMem, 10);

    /* Заполнение блока памяти */
    pIMem->pVTbl->Fill(pIMem, name, 'a', 9);

    /* Получение интерфейса со стандартным вводом/выводом */
    pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdIOC89, 0, &IID_IEcoStdIOC89, (void **)&pIStdIO);
    if (result != 0 && pIStdIO == 0) {
        /* Освобождение интерфейсов в случае ошибки */
        goto Release;
    }
}

```

```

/* Получение интерфейса для работы со строкой */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStringC89, 0, &IID_IEcoStringC89, (void**) &pIString);
if (result != 0 && pIString == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса для работы со временем */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoTimeC89, 0, &IID_IEcoTimeC89, (void**) &pITime);
if (result != 0 && pITime == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdLibC89, 0, &IID_IEcoStdLibC89, (void**) &pIStdLib);
if (result != 0 && pIStdLib == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Тестирование методов интерфейса */
test_atof(pIStdLib, pIStdIO, pIString);
test_atoi(pIStdLib, pIStdIO, pIString);
test_atol(pIStdLib, pIStdIO, pIString);
test_strtod(pIStdLib, pIStdIO);
test_strtol(pIStdLib, pIStdIO);
test_strtoul(pIStdLib, pIStdIO);
test_rand(pIStdLib, pIStdIO, pITime);
test_srand(pIStdLib, pIStdIO, pITime);
test_calloc(pIStdLib, pIStdIO);
test_free(pIStdLib, pIStdIO, pIString);
test_malloc(pIStdLib, pIStdIO, pIString);
test_realloc(pIStdLib, pIStdIO, pIString);
test_abort(pIStdLib, pIStdIO);
test_atexit(pIStdLib, pIStdIO);
test_exit(pIStdLib, pIStdIO);
test_getenv(pIStdLib, pIStdIO);
test_system(pIStdLib, pIStdIO, pIString);
test_bsearch(pIStdLib, pIStdIO);
test_qsort(pIStdLib, pIStdIO);
test_abs(pIStdLib, pIStdIO);
test_div(pIStdLib, pIStdIO);
test_labs(pIStdLib, pIStdIO);
test_ldiv(pIStdLib, pIStdIO);
test_mblen(pIStdLib, pIStdIO);
test_mbtowc(pIStdLib, pIStdIO, pIString);
test_wctomb(pIStdLib, pIStdIO);
test_mbstowcs(pIStdLib, pIStdIO);
test_wcstombs(pIStdLib, pIStdIO);

/* Освобождение блока памяти */
pIMem->pVTbl->Free(pIMem, name);

```

Release:

```

/* Освобождение интерфейса для работы с интерфейсной шиной */
if (pIBus != 0) {

```



```

    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение тестируемого интерфейса */
if (pIStdLib != 0) {
    pIStdLib->pVTbl->Release(pIStdLib);
}

/* Освобождение интерфейса работы со стандартным вводом/выводом */
if (pIStdIO != 0) {
    pIStdIO->pVTbl->Release(pIStdIO);
}

/* Освобождение интерфейса работы со временем */
if (pITime != 0) {
    pITime->pVTbl->Release(pITime);
}

/* Освобождение интерфейса работы со строкой */
if (pIString != 0) {
    pIString->pVTbl->Release(pIString);
}

/* Освобождение системного интерфейса */
if (pISys != 0) {
    pISys->pVTbl->Release(pISys);
}

return result;
}

bool_t test_atof(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString) {
    float val;
    char str[20];

    pIString->pVTbl->strcpy(pIString, str, "98993489");
    val = (float)pIStdLib->pVTbl->atof(pIStdLib, str);
    pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Float value = %f\n", str, val);

    pIString->pVTbl->strcpy(pIString, str, "peerf.me");
    val = (float)pIStdLib->pVTbl->atof(pIStdLib, str);
    pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Float value = %f\n", str, val);

    return 1;
}

bool_t test_atoi(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString) {
    int val;
    char str[20];

    pIString->pVTbl->strcpy(pIString, str, "98993489");
    val = pIStdLib->pVTbl->atoi(pIStdLib, str);

```

```

pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Int value = %d\n", str, val);

pIString->pVTbl->strcpy(pIString, str, "peerf.me");
val = pIStdLib->pVTbl->atoi(pIStdLib, str);
pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Int value = %d\n", str, val);

return 1;
}

bool_t test_atol(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO, IEcoStringC89* pIString) {
    long val;
    char str[20];

    pIString->pVTbl->strcpy(pIString, str, "98993489");
    val = pIStdLib->pVTbl->atol(pIStdLib, str);
    pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Long value = %ld\n", str, val);

    pIString->pVTbl->strcpy(pIString, str, "peerf.me");
    val = pIStdLib->pVTbl->atol(pIStdLib, str);
    pIStdIO->pVTbl->printf(pIStdIO, "String value = %s, Long value = %ld\n", str, val);

    return 1;
}

bool_t test_strtod(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    char str[30] = "20.30300 This is test";
    char *ptr;
    double ret;

    ret = pIStdLib->pVTbl->strtod(pIStdLib, str, &ptr);
    pIStdIO->pVTbl->printf(pIStdIO, "The number(double) is %lf\n", ret);
    pIStdIO->pVTbl->printf(pIStdIO, "String part is |%s|\n", ptr);

    return 1;
}

bool_t test_strtol(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    char str[30] = "2030300 This is test";
    char *ptr;
    long ret;

    ret = pIStdLib->pVTbl->strtol(pIStdLib, str, &ptr, 10);
    pIStdIO->pVTbl->printf(pIStdIO, "The number(unsigned long integer) is %ld\n", ret);
    pIStdIO->pVTbl->printf(pIStdIO, "String part is |%s|\n", ptr);

    return 1;
}

bool_t test_strtoul(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    char str[30] = "2030300 This is test";
    char *ptr;
    long ret;

    ret = pIStdLib->pVTbl->strtoul(pIStdLib, str, &ptr, 10);
    pIStdIO->pVTbl->printf(pIStdIO, "The number(unsigned long integer) is %lu\n", ret);
    pIStdIO->pVTbl->printf(pIStdIO, "String part is |%s|\n", ptr);

    return 1;
}

```

```

bool_t test_rand(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoTimeC89* pITime) {
    int i, n;
    time_t t;

    n = 5;

    /* Intializes random number generator */
    pIStdLib->pVTbl->srand(pIStdLib, (unsigned) pITime->pVTbl->time(pITime, &t));

    /* Print 5 random numbers from 0 to 49 */
    for( i = 0 ; i < n ; i++ ) {
        pIStdIO->pVTbl->printf(pIStdIO, "%d\n", pIStdLib->pVTbl->rand(pIStdLib) % 50);
    }

    return 1;
}

bool_t test_srand(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoTimeC89* pITime) {
    int i, n;
    time_t t;

    n = 5;

    /* Intializes random number generator */
    pIStdLib->pVTbl->srand(pIStdLib, (unsigned) time(&t));

    /* Print 5 random numbers from 0 to 50 */
    for( i = 0 ; i < n ; i++ ) {
        pIStdIO->pVTbl->printf(pIStdIO, "%d\n", pIStdLib->pVTbl->rand(pIStdLib) % 50);
    }

    return 1;
}

bool_t test_calloc(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO) {
    int i, n;
    int *a;

    pIStdIO->pVTbl->printf(pIStdIO, "Number of elements to be entered:");
    pIStdIO->pVTbl->scanf(pIStdIO, "%d",&n);

    a = (int*)pIStdLib->pVTbl->calloc(pIStdLib, n, sizeof(int));
    pIStdIO->pVTbl->printf(pIStdIO, "Enter %d numbers:\n",n);
    for( i=0 ; i < n ; i++ ) {
        pIStdIO->pVTbl->scanf(pIStdIO, "%d",&a[i]);
    }

    pIStdIO->pVTbl->printf(pIStdIO, "The numbers entered are: ");
    for( i=0 ; i < n ; i++ ) {
        pIStdIO->pVTbl->printf(pIStdIO, "%d ",a[i]);
    }
    pIStdLib->pVTbl->free(pIStdLib, a);

    return 1;
}

bool_t test_free(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoStringC89* pIStrng) {

```

```

char *str;

/* Initial memory allocation */
str = (char *)pIStdLib->pVTbl->malloc(pIStdLib, 15);
pIString->pVTbl->strcpy(pIString, str, "peerf");
pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

/* Reallocating memory */
str = (char *)pIStdLib->pVTbl->realloc(pIStdLib, str, 25);
pIString->pVTbl->strcat(pIString, str, ".me");
pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

/* Deallocate allocated memory */
pIStdLib->pVTbl->free(pIStdLib, str);
return 1;
}

bool_t test_malloc(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO, IEcoStringC89* pIString) {
    char *str;

    /* Initial memory allocation */
    str = (char *)pIStdLib->pVTbl->malloc(pIStdLib, 15);
    pIString->pVTbl->strcpy(pIString, str, "peerf");
    pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

    /* Reallocating memory */
    str = (char *)pIStdLib->pVTbl->realloc(pIStdLib, str, 25);
    pIString->pVTbl->strcat(pIString, str, ".me");
    pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

    pIStdLib->pVTbl->free(pIStdLib, str);
    return 1;
}

bool_t test_realloc(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO, IEcoStringC89* pIString) {
    char *str;

    /* Initial memory allocation */
    str = (char *)pIStdLib->pVTbl->malloc(pIStdLib, 15);
    pIString->pVTbl->strcpy(pIString, str, "peerf");
    pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

    /* Reallocating memory */
    str = (char *)pIStdLib->pVTbl->realloc(pIStdLib, str, 25);
    pIString->pVTbl->strcat(pIString, str, ".me");
    pIStdIO->pVTbl->printf(pIStdIO, "String = %s, Address = %u\n", str, str);

    pIStdLib->pVTbl->free(pIStdLib, str);

    return 1;
}

bool_t test_abort(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    FILE *fp;

    pIStdIO->pVTbl->printf(pIStdIO, "Going to open nofile.txt\n");
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "nofile.txt", "r" );
    if(fp == NULL) {

```

```

        pIStdIO->pVTbl->printf(pIStdIO, "Going to abort the program\n");
        //pIStdLib->pVTbl->abort(pIStdLib);
    }
    pIStdIO->pVTbl->printf(pIStdIO, "Going to close nofile.txt\n");
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_atexit(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO) {
    /* register the termination function */
    pIStdLib->pVTbl->atexit(pIStdLib, functionA );

    pIStdIO->pVTbl->printf(pIStdIO, "Starting main program...\n");

    pIStdIO->pVTbl->printf(pIStdIO, "Exiting main program...\n");
    return 1;
}

bool_t test_exit(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO) {
    pIStdIO->pVTbl->printf(pIStdIO, "Start of the program....\n");

    pIStdIO->pVTbl->printf(pIStdIO, "Exiting the program....\n");
    //pIStdLib->pVTbl->exit(pIStdLib, 0);

    pIStdIO->pVTbl->printf(pIStdIO, "End of the program....\n");

    return 1;
}

bool_t test_getenv(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO) {
    pIStdIO->pVTbl->printf(pIStdIO, "PATH : %s\n", pIStdLib->pVTbl->getenv(pIStdLib, "PATH"));
    pIStdIO->pVTbl->printf(pIStdIO, "HOME : %s\n", pIStdLib->pVTbl->getenv(pIStdLib, "HOME"));
    pIStdIO->pVTbl->printf(pIStdIO, "ROOT : %s\n", pIStdLib->pVTbl->getenv(pIStdLib, "ROOT"));

    return 1;
}

bool_t test_system(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString) {
    char command[50];

    pIString->pVTbl->strcpy(pIString, command, "ls -l" );
    pIStdLib->pVTbl->system(pIStdLib, command);
    return 1;
}

bool_t test_bsearch(IEcoStdLibC89* pIStdLib, IEcoStdIOc89* pIStdIO) {
    int *item;
    int key = 2;

    /* using bsearch() to find value 32 in the array */
    item = (int*)pIStdLib->pVTbl->bsearch (pIStdLib, &key, values, 5, sizeof (int), cmpfunc);
    if( item != NULL ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Found item = %d\n", *item);
    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "Item = %d could not be found\n", key);
    }
    return 1;
}

```

```

bool_t test_qsort(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    int n;

    pIStdIO->pVTbl->printf(pIStdIO, "Before sorting the list is: \n");
    for( n = 0 ; n < 5; n++ ) {
        pIStdIO->pVTbl->printf(pIStdIO, "%d ", values[n]);
    }

    pIStdLib->pVTbl->qsort(pIStdLib, values, 5, sizeof(int), cmpfunc);

    pIStdIO->pVTbl->printf(pIStdIO, "\nAfter sorting the list is: \n");
    for( n = 0 ; n < 5; n++ ) {
        pIStdIO->pVTbl->printf(pIStdIO, "%d ", values[n]);
    }

    return 1;
}

bool_t test_abs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    int a, b;

    a = pIStdLib->pVTbl->abs(pIStdLib, 5);
    pIStdIO->pVTbl->printf(pIStdIO, "value of a = %d\n", a);

    b = pIStdLib->pVTbl->abs(pIStdLib, -10);
    pIStdIO->pVTbl->printf(pIStdIO, "value of b = %d\n", b);

    return 1;
}

bool_t test_div(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    div_t output;

    output = pIStdLib->pVTbl->div(pIStdLib, 27, 4);
    pIStdIO->pVTbl->printf(pIStdIO, "Quotient part of (27/ 4) = %d\n", output.quot);
    pIStdIO->pVTbl->printf(pIStdIO, "Remainder part of (27/4) = %d\n", output.rem);

    output = pIStdLib->pVTbl->div(pIStdLib, 27, 3);
    pIStdIO->pVTbl->printf(pIStdIO, "Quotient part of (27/ 3) = %d\n", output.quot);
    pIStdIO->pVTbl->printf(pIStdIO, "Remainder part of (27/3) = %d\n", output.rem);

    return 1;
}

bool_t test_labs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    long int a,b;

    a = pIStdLib->pVTbl->labs(pIStdLib, 65987L);
    pIStdIO->pVTbl->printf(pIStdIO, "Value of a = %ld\n", a);

    b = pIStdLib->pVTbl->labs(pIStdLib, -1005090L);
    pIStdIO->pVTbl->printf(pIStdIO, "Value of b = %ld\n", b);

    return 1;
}

bool_t test_ldiv(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {

```

```

ldiv_t output;

output = pIStdLib->pVTbl->ldiv(pIStdLib, 100000L, 30000L);

pIStdIO->pVTbl->printf(pIStdIO, "Quotient = %ld\n", output.quot);

pIStdIO->pVTbl->printf(pIStdIO, "Remainder = %ld\n", output.rem);

return 1;
}

bool_t test_mblen(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    int len;
    char *pmbnull = NULL;
    char *pmb = (char *)pIStdLib->pVTbl->malloc(pIStdLib, MB_CUR_MAX );
    wchar_t *pwc = L"Hi";
    wchar_t *pwcs = (wchar_t *)pIStdLib->pVTbl->malloc(pIStdLib, sizeof( wchar_t ));

    pIStdIO->pVTbl->printf(pIStdIO, "Converting to multibyte string\n");
    len = pIStdLib->pVTbl->wcstombs(pIStdLib, pmb, pwc, MB_CUR_MAX);
    pIStdIO->pVTbl->printf(pIStdIO, "Characters converted %d\n", len);
    pIStdIO->pVTbl->printf(pIStdIO, "Hex value of first multibyte character: %#.4x\n", pmb);

    len = pIStdLib->pVTbl->mblen(pIStdLib, pmb, MB_CUR_MAX );
    pIStdIO->pVTbl->printf(pIStdIO, "Length in bytes of multibyte character %x: %u\n", pmb, len );

    pmb = NULL;

    len = pIStdLib->pVTbl->mblen(pIStdLib, pmb, MB_CUR_MAX );
    pIStdIO->pVTbl->printf(pIStdIO, "Length in bytes of multibyte character %x: %u\n", pmb, len );

    pIStdLib->pVTbl->free(pIStdLib, pmb);
    pIStdLib->pVTbl->free(pIStdLib, pwcs);

    return 1;
}

bool_t test_mbtowc(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO, IEcoStringC89* pIString) {
    char *str = "This is peerf.me";
    wchar_t mb[100];
    int len;

    len = pIStdLib->pVTbl->mblen(pIStdLib, NULL, MB_CUR_MAX);

    //pIStdLib->pVTbl->mbtowc(pIStdLib, mb, str, len*pIString->pVTbl->strlen(pIString, str) );

    //pIStdIO->pVTbl->wprintf(pIStdIO, L"%ls \n", mb );
    return 1;
}

bool_t test_wctomb(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    int i;
    wchar_t wc = L'a';
    char *pmbnull = NULL;
    char *pmb = (char *)pIStdLib->pVTbl->malloc(pIStdLib, sizeof( char ));

    pIStdIO->pVTbl->printf(pIStdIO, "Converting wide character:\n");
    i = pIStdLib->pVTbl->wctomb(pIStdLib, pmb, wc );

```

```
pIStdIO->pVTbl->printf(pIStdIO, "Characters converted: %u\n", i);
pIStdIO->pVTbl->printf(pIStdIO, "Multibyte character: %.1s\n", pmb);

pIStdIO->pVTbl->printf(pIStdIO, "Trying to convert when target is NULL:\n");
i = pIStdLib->pVTbl->wctomb(pIStdLib, pmbnull, wc );
pIStdIO->pVTbl->printf(pIStdIO, "Characters converted: %u\n", i);
/* this will not print any value */
pIStdIO->pVTbl->printf(pIStdIO, "Multibyte character: %.1s\n", pmbnull);

pIStdLib->pVTbl->free(pIStdLib, pmb);

return 1;
}
```

```
bool_t test_mbstowcs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    int len;
    char *pmbnull = NULL;
    char *pmb = (char *)pIStdLib->pVTbl->malloc(pIStdLib, MB_CUR_MAX );
    wchar_t *pwc = L"Hi";
    wchar_t *pwcs = (wchar_t *)pIStdLib->pVTbl->malloc(pIStdLib, sizeof( wchar_t ));

    pIStdIO->pVTbl->printf(pIStdIO, "Converting to multibyte string\n");
    len = pIStdLib->pVTbl->wcstombs(pIStdLib, pmb, pwc, MB_CUR_MAX);
    pIStdIO->pVTbl->printf(pIStdIO, "Characters converted %d\n", len);
    pIStdIO->pVTbl->printf(pIStdIO, "Hex value of first multibyte character: %#.4x\n", pmb);

    pIStdIO->pVTbl->printf(pIStdIO, "Converting back to Wide-Character string\n");
    len = pIStdLib->pVTbl->mbstowcs(pIStdLib, pwcs, pmb, MB_CUR_MAX);
    pIStdIO->pVTbl->printf(pIStdIO, "Characters converted %d\n", len);
    pIStdIO->pVTbl->printf(pIStdIO, "Hex value of first wide character %#.4x\n\n", pwcs);

    pIStdLib->pVTbl->free(pIStdLib, pmb);
    pIStdLib->pVTbl->free(pIStdLib, pwcs);

    return 1;
}
```

```
bool_t test_wcstombs(IEcoStdLibC89* pIStdLib, IEcoStdIOC89* pIStdIO) {
    size_t ret;
    char *MB = (char *)pIStdLib->pVTbl->malloc(pIStdLib, BUFFER_SIZE );
    wchar_t *WC = L"https://www.peerf.me";

    /* converting wide-character string */
    ret = pIStdLib->pVTbl->wcstombs(pIStdLib, MB, WC, BUFFER_SIZE);

    pIStdIO->pVTbl->printf(pIStdIO, "Characters converted = %u\n", ret);
    pIStdIO->pVTbl->printf(pIStdIO, "Multibyte character = %s\n\n", MB);

    pIStdLib->pVTbl->free(pIStdLib, MB);

    return 1;
}
```